MSML/MSAI 605
Milestone 4 (Detailed)

What Milestone 4 is testing

Milestone 4 is the final audit, profiling, and release milestone for your face verification project. In Milestone 1, you built the deterministic plumbing. In Milestone 2, you added the evaluation and iteration loop. In Milestone 3, you turned the verifier into an embedding-based inference system packaged for deployment. In Milestone 4, you now take that near-complete system and finish it responsibly.

This milestone is not mainly about adding another major model change. It is about documenting the final system clearly, profiling how it behaves on real hardware, stating where it works and where it can fail, and making the final release reproducible from a clean clone.

- Responsible ML audit, meaning you describe intended use, limitations, failure modes, and fairness-related risks in a professional System Card.
- Hardware-aware profiling, meaning you measure preprocessing, embedding, and scoring latency separately and report how runtime changes across conditions such as batch size.
- Final system alignment, meaning the reported operating threshold, metrics, CLI behavior, and documentation all refer to the same final system version.
- Reproducibility, meaning a grader can follow exact commands to reproduce the core results and run the Dockerized CLI from a fresh clone.
- Release discipline, meaning the final tag, README, System Card, profiling report, and reproducibility checklist all match one another.

What you must submit in Canvas

- Your Git repository link.
- Your final Git tag (example: v1.0-final).
- The link or path in the repository to the final System Card, profiling report, and reproducibility checklist if those locations are not obvious from the README.

Definition (quick checklist)

- The repository still has a clean structure, a pinned environment, and a README with a brief project overview plus a clear How to run section.
- You provide a final System Card, about 1 to 6 pages, covering intended use, limitations, failure modes, key metrics, operating threshold, fairness-risk discussion, and operational constraints.
- You provide a profiling report, notebook, or short writeup that includes a latency breakdown for preprocessing vs embedding vs scoring, along with batch-size sensitivity.
- You provide a CPU baseline profile. A GPU comparison is optional, but if you include one, describe the hardware and methodology clearly.
- You provide a reproducibility checklist with exact commands to reproduce the core results and run the Dockerized CLI.
- The final release keeps the Milestone 3 inference path runnable and aligned with the reported system version.
- From a fresh clone, a grader can reproduce the key outputs and inspect the final release artifacts.
- You created and pushed a final tag such as v1.0-final that points to the reproducible release commit.

1. How Milestone 4 fits into this project

A common mistake in Milestone 4 is to treat it as only a writing exercise. That is not what the project plan intends. Milestone 4 is the final systems-quality milestone. It asks you to take the working verifier from Milestone 3 and make its behavior, risks, constraints, and runtime characteristics explicit.

- Milestone 1 built the deterministic system backbone.
- Milestone 2 built the thresholding, evaluation, and iteration loop.
- Milestone 3 upgraded the representation to embeddings and exposed deployable inference.
- Milestone 4 now audits, profiles, documents, and finalizes that complete system.

So, in Milestone 4, the central question is not "Can I add one more feature?" The real question is "Can I present my verifier as a complete, reproducible system with clear operating boundaries, believable measurements, and responsible documentation?"

Milestone 4 should therefore stabilize the project rather than destabilize it. The goal is a coherent final release, not a last-minute collection of disconnected changes.

The table below summarizes what each milestone contributes to the project flow.

Milestone	Main contribution to the project
Milestone 1	Deterministic ingestion, saved pairs, reproducible structure, and fast vectorized scoring.
Milestone 2	Threshold calibration, tracked runs, error analysis, data-centric iteration, validation checks, and tests.
Milestone 3	Embedding-based inference, Docker packaging, CLI interface, and concurrency or load testing.
Milestone 4	System Card, limitation and fairness-risk analysis, hardware-aware profiling, reproducibility checklist, and final release alignment.

2. Important Milestone 4 terms, in project context

Before you implement Milestone 4, make sure the terms below are clear in the context of this project.

Term	What it means in this milestone
System Card	A short professional document describing intended use, operating threshold, key metrics, limitations, failure modes, fairness-related risks, and deployment constraints for the final verifier.
Intended use	The use cases the system is meant to support. You should also state clearly what uses are out of scope.
Failure mode	A pattern of inputs or conditions under which the verifier may give unreliable decisions or confidence values.
Operational constraint	A practical restriction such as latency budget, hardware assumptions, image quality assumptions, or supported input format.
Fairness-risk discussion	A responsible description of where uneven performance or harmful misuse could arise. Do not invent claims that your data does not support.
Hardware-aware profiling	Measuring runtime behavior with attention to hardware context, including stage-wise latency and the effect of batch size or optional GPU use.
CPU baseline	A required profile of the system on CPU so the grader has a common reference point.
Batch-size sensitivity	How latency and throughput change when the number of processed pairs changes.
Reproducibility checklist	A compact set of exact commands and artifact paths that lets a grader recreate the key results from a clean clone.

3. Recommended repository additions for Milestone 4

You may keep the same overall repository layout from Milestones 1 to 3. Milestone 4 usually adds clearer places for the final System Card, profiling artifacts, and the reproducibility checklist. The exact folder names can vary, but the responsibilities should be clear.

Area	What belongs there
README.md	Final project overview, design summary, Docker or CLI instructions, artifact locations, and the final reproducibility path.
reports/	Final System Card PDF, profiling writeup or notebook export, and any lightweight supporting figures.
src/	The final importable inference, profiling helpers, and any utilities still needed for evaluation or system reporting.
scripts/	Runnable entrypoints for profiling, Docker or CLI inference, final evaluation reproduction, and checklist validation.
configs/	Threshold settings, final inference defaults, profiling settings, and any documented environment assumptions.
tests/	Relevant unit tests and smoke tests that still apply to the final release.
artifacts/ or outputs/	Optional lightweight profiling summaries, metric tables, and logs that help the grader inspect the final system.

4. .gitignore expectations for Milestone 4

- Ignore downloaded dataset caches, large generated outputs, heavy profiling dumps, and local-only logs unless you intentionally include a small committed example for grading.
- Ignore local virtual environments, caches, notebook checkpoints, operating system files, and temporary Docker or hardware-profiling artifacts.
- Commit the code, configs, README, reports, Dockerfile, tests, and only lightweight evidence that is genuinely helpful for grading.
- Do not commit large local outputs simply because they were produced during experimentation.

5. Step-by-step: Milestone 4 tasks

Step 5.1 Freeze the final system version

Before writing the System Card or running the final profiles, decide which exact system version you are finalizing. Milestone 4 is about consistency. If the System Card refers to one threshold, the CLI uses another, and the profiling notebook measures a third configuration, the milestone becomes hard to trust.

- Choose the final embedding-based system version that will be documented and tagged.
- Make sure the operating threshold for that final system is clearly recorded.
- Keep the final release aligned with the Dockerized CLI path from Milestone 3.

Step 5.2 Confirm the final operating threshold and key metrics

Milestone 4 should not contradict Milestone 2 or Milestone 3. The operating threshold used in the System Card should be the threshold actually used by the final system version. If you changed the representation or score pipeline in Milestone 3, make sure the documented threshold belongs to that embedding-based system, not to the old baseline.

- State what split and rule were used to choose the final threshold.
- Summarize the key verification metrics at that threshold.
- Make sure the metrics reported in the System Card, README, and supporting artifacts are consistent.

Step 5.3 Write the final System Card

The System Card is the main professional document in Milestone 4. It should describe what the system is, what it is intended to do, where it can fail, what threshold it uses, and what practical constraints apply when running it.

- Keep the System Card focused and readable. The project plan allows about 1 to 6 pages.
- Use plain language, but be specific enough that a grader can understand the true operating assumptions of your verifier.
- Do not treat the System Card as a marketing document. It should include real limitations and risks.

Step 5.4 Include the right sections in the System Card

A strong System Card usually contains the sections below. The exact headings can vary, but the content should be present.

- System overview and pipeline summary.
- Intended use and explicitly out-of-scope uses.
- Data summary and any important data limitations.
- Operating threshold and key metrics at that threshold.
- Observed failure modes and limitations.
- Fairness-related risks and misuse concerns.
- Operational constraints such as expected input format, latency assumptions, or hardware requirements.
- Reproducibility pointer to the README and final tag.

Step 5.5 Discuss limitations and fairness-related risks responsibly

This is one of the most important parts of Milestone 4. Because the project is a face verification system, you should not write only about technical performance. You should also describe where harm, misuse, or uneven performance could occur.

- If you do not have reliable demographic metadata, do not invent unsupported fairness claims.
- You may still discuss fairness-related risks responsibly by describing known risk categories such as image quality differences, pose and lighting variation, occlusion, visually similar different-identity pairs, and possible uneven performance across populations.
- If you choose to include any subgroup-style analysis, define the groups explicitly, state the limitations of the grouping method, and avoid overclaiming.

Step 5.6 Run hardware-aware profiling

The project plan explicitly asks for a profiling report with a latency breakdown of preprocessing vs embedding vs scoring. Milestone 4 is therefore the place where you measure the runtime behavior of the complete embedding-based system you finalized in Milestone 3.

- Measure preprocessing latency.
- Measure embedding-generation latency.
- Measure similarity-scoring latency.
- Report end-to-end latency as well.
- Use consistent timing methodology and document what you measured.

Step 5.7 Provide a CPU baseline and optional GPU comparison

A CPU baseline is required because it gives the grader a common reference point. A GPU comparison is optional. If you include a GPU result, it should be clearly labeled as optional supplemental evidence rather than a replacement for the CPU baseline.

- State the hardware used for the CPU baseline clearly.
- If you include GPU measurements, state the device and software stack clearly.
- Do not compare CPU and GPU results casually without describing the measurement setup.

Step 5.8 Analyze batch-size sensitivity

Milestone 4 also asks for batch-size sensitivity. This means your profiling report should show how runtime behavior changes when you vary the amount of work processed together.

- Use at least a small set of batch sizes that is practical for your system.
- Report how latency and, if useful, throughput change across those sizes.
- Explain the tradeoff briefly instead of only dumping numbers.

Step 5.9 Prepare the profiling report

The profiling report can be a notebook or a short writeup. Its goal is not to be long. Its goal is to let the grader understand how the final system behaves and which stage dominates latency.

- Describe the measurement environment and assumptions.
- Include the per-stage latency breakdown.
- Include batch-size sensitivity.
- Make it obvious where the CPU baseline is shown.
- If you include GPU results, keep them clearly separated from the required CPU baseline.

Step 5.10 Prepare the reproducibility checklist

The reproducibility checklist should be concise and operational. A grader should be able to follow it without reading your whole repository history.

- List exact commands to set up the environment or build the Docker image.
- List exact commands to run the CLI and reproduce the core results.
- List where the System Card and profiling report are located.
- List the final Git tag and any key config files needed to reproduce the system behavior.

Step 5.11 Finalize the README

By Milestone 4, the README should read like the entry point to the final release. It should not force the grader to guess where the important artifacts are.

- Summarize the final system briefly.
- Point to the System Card, profiling report, and reproducibility checklist.
- Include copy-pastable environment, Docker, CLI, and testing commands.
- Make the final artifact locations obvious.

Step 5.12 Run the clean-clone final check

Before tagging the final release, test everything from a fresh clone. This is the same principle used in earlier milestones, but it matters even more now because multiple final artifacts have to agree with one another.

- Follow your own README and reproducibility checklist exactly.
- Confirm that the Dockerized CLI runs.

- Confirm that the System Card and profiling report match the tagged system version.
- Confirm that the key outputs and artifact paths are easy to find.

Step 5.13 Tag the final release only after alignment passes

Create the final tag only after the System Card, profiling report, reproducibility checklist, README, CLI behavior, and supporting artifacts are aligned with one another.

- Use a final tag such as v1.0-final.
- Make sure the tag points to the exact reproducible commit.
- Do not tag a commit first and fix the final artifacts later.

6. System Card guidance for this project

The System Card is the most visible deliverable in Milestone 4. It should reflect the final deployed verifier, not an earlier experiment. A useful way to think about it is that someone reading only the System Card should still be able to understand what the system does, where it should and should not be used, and what operating assumptions matter.

System Card section	What it should cover
System overview	What the verifier does, what inputs it accepts, and the high-level pipeline.
Intended use	The cases the system is meant to support, plus clear non-intended or out-of-scope uses.
Data summary	A concise description of the data source and any major data limitations relevant to interpretation.
Operating threshold and metrics	The selected operating threshold and the key reported metrics for the final system version.
Failure modes and limitations	Where the system tends to break down or become less reliable.
Fairness-risk discussion	Potential uneven performance, misuse risks, or ethical concerns, stated carefully and honestly.
Operational constraints	Hardware, latency, input-format, or deployment assumptions that affect reliable use.
Reproducibility pointer	Where to find the README, commands, final tag, and supporting reports.

7. Profiling report guidance for this project

A strong profiling report is short but specific. It should let the grader see how the final system behaves in practice and which stage contributes most to runtime.

Profiling element	Expected content
Measurement environment	CPU details are required. Include OS, major software assumptions, and any optional GPU details if used.
Per-stage latency	Separate preprocessing, embedding, and scoring. End-to-end latency is also useful.
Methodology	How many repetitions or runs were used, how timing was measured, and what inputs were used.
Batch-size sensitivity	How latency or throughput changes across a practical range of batch sizes.
CPU baseline	A clear baseline result that the grader can interpret without relying on GPU access.
Optional GPU comparison	Supplemental results only, clearly labeled and methodologically described.
Interpretation	A short explanation of which stage dominates and what tradeoffs the measurements suggest.

8. Recommended README and reproducibility checklist content

The README and reproducibility checklist should work together. The README can give the broader overview, while the checklist gives a minimal exact path to reproduce the final system and its core outputs.

- Final project overview and pipeline summary.
- Location of the System Card, profiling report, and reproducibility checklist.
- Copy-pastable environment setup or Docker build commands.
- Exact commands for running the CLI on a sample pair or sample batch.
- Exact commands for reproducing the key profiling artifact or summary.
- The final tag and any key config files needed to reproduce the final system behavior.

9. Common mistakes to avoid

- Treating Milestone 4 as only a writing task and forgetting that the reported system must still run.
- Changing the final model, threshold, or inference path after writing the System Card, so the artifacts no longer match.
- Writing a fairness section that makes unsupported claims not backed by your data or analysis.
- Reporting only end-to-end latency and forgetting the required preprocessing vs embedding vs scoring breakdown.
- Including GPU numbers without a required CPU baseline.
- Publishing a reproducibility checklist with vague or incomplete commands.
- Leaving the README, System Card, profiling report, and final tag out of sync.
- Tagging the final release before the clean-clone verification path has been tested.

10. One acceptable Milestone 4 implementation plan

There is more than one valid way to complete Milestone 4. The phases below describe one clean and reasonable path.

Phase	Main goal
Phase A	Freeze the Milestone 3 system version that will become the final release.
Phase B	Confirm the operating threshold, final metrics, and release-aligned inference path.
Phase C	Write the System Card with intended use, limitations, failure modes, and fairness-related risks.
Phase D	Run CPU profiling, optional GPU profiling, and batch-size sensitivity measurements.
Phase E	Prepare the profiling report, reproducibility checklist, and final README alignment.
Phase F	Run the clean-clone final check and tag the exact reproducible commit as v1.0-final.

11. Suggested Milestone 4 artifacts

- Final System Card PDF in reports/.
- Profiling report notebook, PDF, or short writeup in reports/.
- Reproducibility checklist in reports/ or the repository root.
- README with final project overview, artifact locations, and copy-pastable commands.
- Dockerfile and final CLI entrypoint from the Milestone 3 system.
- Any small profiling summary tables or figures that help the grader inspect the final release.
- A final tag such as v1.0-final that points to the reproducible release commit.

12. Final summary

Milestone 4 is where your face verification project becomes a final professional release. Carry forward the deterministic backbone from Milestone 1, the evaluation discipline from Milestone 2, and the embedding-based deployable inference system from Milestone 3. Then audit that final system responsibly, profile how it behaves on real hardware, document its limits and risks clearly, and make the final release reproducible from a clean clone.

By the end of this milestone, a grader should be able to read your System Card, inspect your profiling report, run the Dockerized CLI, follow your reproducibility checklist, and confirm that the final tagged release matches the documented system.